



Modular Design of KEM-Based Authenticated Key Exchange

Colin Boyd, Bor de Kock^(✉), and Lise Millerjord

NTNU – Norwegian University of Science and Technology, Trondheim, Norway
{colin.boyd,bor.dekock,lise.millerjord}@ntnu.no

Abstract. A key encapsulation mechanism (KEM) is a basic building block for key exchange which must be combined with long-term keys in order to achieve authenticated key exchange (AKE). Although several KEM-based AKE protocols have been proposed, KEM-based modular building blocks are not available. We provide a KEM-based authenticator and a KEM-based protocol in the Authenticated Links model (AM), in the terminology of Canetti and Krawczyk (2001). Using these building blocks we achieve a set of generic AKE protocols. By instantiating these with post quantum primitives we are able to propose several new post-quantum secure AKE protocols.

1 Introduction

Authenticated key exchange (AKE) is a fundamental tool for establishing secure communications. An important component in the design of AKE protocols is Diffie–Hellman (DH) key exchange, due to its versatility and potential for providing security properties such as forward secrecy. Today many real-world AKE protocols are based on DH implementations, typically in elliptic curve groups; examples include TLS, IPsec, WireGuard and the generic Noise Framework.

The looming threat of quantum computers has brought about an increasingly pressing need to find post-quantum secure replacements for DH, which itself is well known to be broken by Shor’s quantum algorithm for finding discrete logarithms [5]. In the absence of many promising candidates for a post-quantum secure direct DH replacement, designs for post-quantum AKE have tended to make use of key encapsulation mechanisms (KEM). This approach aligns well with the research literature where many post-quantum candidate KEMs have been proposed and also with the prominent NIST post-quantum cryptography competition [1] which requests primitives of only two types, namely a KEM or a digital signature. Although DH can be framed as a KEM, DH has special properties which prevent KEMs from being used as a drop-in replacement for DH. For example, DH has the property that two parties can generate their DH

Boyd and Millerjord are supported by the Research Council of Norway under Project No. 288545. Author list in alphabetical order; see <https://www.ams.org/profession/leaders/culture/CultureStatement04.pdf>. A full version of this work is available on the IACR ePrint Archive under number 2023/167 [8].

shares completely independently; this cannot be achieved in general with KEMs, but rather one party must wait for the other party's input.

Achieving the *authenticated* part of AKE has traditionally been done by applying a digital signature scheme on the messages of a key exchange protocol, but authentication can also be achieved in different ways, which can be advantageous for several reasons: for instance to achieve a speed-up or use less memory, as other works have demonstrated [23].

Although in 2022 the NIST competition for post-quantum secure cryptography (PQ or PQ-crypto) has led to the standardization of several signature schemes, only one KEM was standardized along with an open call for more schemes to be proposed [1]. To achieve authentication without depending on this one scheme is a desirable property. One of the main motivations for our work is to be flexible in the use of cryptographic primitives so that as the security of post-quantum KEMs or signatures becomes better understood, and as new primitives are designed, it is easy to swap in and out different ones.

1.1 Modular Design of AKE

Over the past decade, several key exchange protocols using post-quantum candidate KEMs have been proposed, both authenticated [13] and unauthenticated [7, 14, 22]. Some of these protocols have been proposed based on specific KEM constructions and the security proofs (where available) relate to specific computational assumptions. These are essential constructions for instantiating protocols using abstract primitives, but when using specific constructions as the basis of security for AKE there is a loss of cryptographic agility. Our goal in this work is to design generic AKE protocols where we can be as flexible as possible with regard to choice of specific KEM instantiations and how they are used.

Our protocol designs are based on the modular approach of Bellare, Canetti and Krawczyk [3, 17] (hereafter referred to as BCK98) and Canetti and Krawczyk [9] (hereafter referred to as CK01). This approach entails defining protocols which are secure in a world which is ideally authenticated and then compiling these protocols with *authenticators* to achieve protocols secure in a world where adversaries completely control the network. A brief introduction to this modular approach is given in Sect. 2.2.

A significant benefit of the modular approach is the ability to “mix-and-match” different components and to use different concrete instances of the same component within one protocol instantiation. This leads to a plethora of different concrete protocols with varying performance characteristics.

We remark that there already exist several protocol designs which are generic in the sense that they can use any specific (secure) instance of different cryptographic primitives such as non-interactive key exchange (NIKE), signatures and/or KEMs [4, 15, 19]. However, such designs do not allow generic mixing of different generic primitives as can be done with the modular approach. For example, the modular approach can be used to replace a digital signature authentication method which a MAC-based method in the case that a pre-shared key is available in a particular application.

1.2 Contributions

We regard the following as the main contributions of this paper:

1. We develop a new KEM-based authenticator and prove its security as a valid authenticator in the CK01 definition, relying on the established CCA-security definition for KEMs.
2. We frame the well-known method of using an ephemeral KEM as a DH replacement as a protocol in the authenticated-links model (AM) of CK01 and prove its security in that model, relying on the established CPA-security definition for KEMs.
3. We derive efficient and secure generic AKE protocols which can be instantiated with any appropriately secure KEMs and also matched with other primitives such as signatures. Some of these generic protocols are completely new, allowing new instantiations of concrete protocols.

1.3 Related Work

In 2017, De Saint Guilhem, Smart and Warinschi [12] presented a generic transformation to convert any two-round forward-secret, but only passively secure, key agreement protocol into a three-round authenticated key agreement protocol. Recognising the value of avoiding signatures for authentication in the post-quantum setting, their transformation makes use of generic CCA-secure public key encryption and a secure MAC. While the approach of De Saint Guilhem et al. has clear parallels with ours, they rely on encryption rather than the often more efficient notion of KEMs. Moreover, they do not allow mixing of different authentication methods as we do, nor provide KEM-based concrete passively secure protocols. Furthermore, their proofs require a key derivation function modelled as a random oracle. Interestingly, they dismiss the CK01 modular approach stating that it necessarily results in increased number of rounds; below we will explain why this is not the case.

Several recent works show that KEM-based approaches are suitable for replacing signatures in real-world applications. The KEMTLS protocol of Schwabe, Stebila and Wiggers [23] is for instance a complete reworking of the TLS 1.3 handshake without using signatures, showing that this would in theory require only half the bandwidth compared to a classical approach—with additional improvements to be gained if the public keys are exchanged in advance [24]. Some of these theoretical improvements turned out to be less impactful when looking at a real-world implementation [10].

Using KEM as a building block for AKE is also done in some other purpose-specific works: examples of this include Post-Quantum Noise [2], FSXY [15] and Post-Quantum WireGuard [18]. These are generic in the sense that any suitable KEMs can be used, but they do not allow the flexibility of different authenticators that we obtain. Specifically, they do not provide re-usable and interchangeable components for passive security and for authentication.

There exist many formal security models for AKE, amongst which several are incomparable [11] in the sense that any one model is often neither stronger nor

weaker than another. The modular approach that we use [9] achieves security in the well-established model known widely as the CK-model. This encompasses fundamental security properties of session key indistinguishability against active attackers who can obtain non-target session keys and adaptively corrupt non-target parties. Forward secrecy is also captured. This model can be adapted [21] if other security properties, such as ephemeral key leakage, are desirable.

2 Background

The main goal of this section is to present the background necessary to understand the modular approach of (Bellare), Canetti and Krawczyk [3,9]. This includes our method to optimise, in a rigorous way, protocols obtained through the approach.

2.1 Standard Definitions

We make use of standard definitions such as KEM, MAC, signatures etc. These can be found in, for example, the textbook of Katz and Lindell [20].

Definition 1. *A key encapsulation mechanism (KEM) is a tuple of PPT algorithms $(\text{Gen}, \text{Encap}, \text{Decap})$ such that:*

1. *The key generation algorithm Gen takes the security parameter 1^n and outputs a public-/private-key pair (sk, pk) : $(sk, pk) \leftarrow \text{Gen}(1^n)$.*
2. *The encapsulation algorithm Encap takes as input a public key pk . It outputs a ciphertext c and a key $k \in \{0, 1\}^{l(n)}$: $(c, k) \leftarrow \text{Encap}(pk)$.*
3. *The deterministic decapsulation algorithm Decap takes as input a private key sk and a ciphertext c , and outputs a key k or the special symbol $\perp$ denoting failure: $k \leftarrow \text{Decap}(sk, c)$.*

We require correctness from the KEM: If $(sk, pk) \leftarrow \text{Gen}(1^n)$ and $(c, k) \leftarrow \text{Encap}_{pk}(1^n)$, and $k' \leftarrow \text{Decap}_{sk}(c)$, then $k' = k$ except with negligible probability.

Furthermore we show the CPA (resp. CCA) indistinguishability experiment(s) for KEMs.

Definition 2. *The CPA resp. CCA indistinguishability experiment proceeds as follows:*

1. *The key generation algorithm is run: $(pk, sk) \leftarrow \text{Gen}(1^n)$.*
2. *The encapsulation algorithm is run: $(c, k) \leftarrow \text{Encap}(pk)$, with $k \in \{0, 1\}^n$.*
3. *A uniform bit $b \in \{0, 1\}$ is chosen. If $b = 0$, set $k' = k$. Otherwise, if $b = 1$, choose a uniform $k' \in \{0, 1\}^n$.*
4. *The experiments outputs (pk, c, k') to $\mathcal{A}$.*

$\mathcal{A}$ is also given access to a decapsulation oracle, $\text{Decap}_{sk}(\cdot)$, but cannot query the decapsulation oracle on the ciphertext c .

5. $\mathcal{A}$ outputs a bit b' . If $b' = b$, $\mathcal{A}$ wins and the experiment outputs 1. Otherwise, $\mathcal{A}$ loses and the experiment outputs 0.

The advantage of the adversary $\mathcal{A}$ in the CPA $\boxed{\text{CCA}}$ experiment is defined to be:

$$\text{Adv}_{\text{KEM}}^{\text{CPA} \boxed{\text{CCA}}}(\mathcal{A}) = 2 \cdot |\Pr[b' = b] - 1/2|.$$

Definition 3. A message authentication code or MAC consists of three probabilistic polynomial-time algorithms (Gen , Mac , MacVer) such that:

1. The key-generation algorithm Gen takes as input the security parameter 1^n and outputs a key k with $|k| \geq n$.
2. The tag-generation algorithm Mac takes as input a key k and a message $m \in \{0, 1\}^*$, and outputs a tag t . Since this algorithm may be randomized, we write this as $t \leftarrow \text{Mac}_k(m)$.
3. The deterministic verification algorithm MacVer takes as input a key k , a message m and a tag t . It outputs a bit b , with $b = 1$ meaning valid and $b = 0$ meaning invalid. We write this as $b := \text{MacVer}_k(m, t)$.

It is required that for every n , every key k and output by $\text{Gen}(1^n)$ and every $m \in \{0, 1\}^*$, it holds that $\text{MacVer}_k(m, \text{Mac}(m)) = 1$.

Definition 4. The existential unforgeability under chosen message attacks (EUF-CMA) experiment for $\text{MAC}(\text{Gen}, \text{Mac}, \text{MacVer})$ proceeds as follows:

1. A key is generated: $k \leftarrow \text{Gen}(1^n)$.
2. The adversary $\mathcal{A}$ gets oracle access to $\text{Mac}_k(\cdot)$. Let Q be the set of all queries $\mathcal{A}$ made to the oracle. The adversary eventually outputs (m, t) .
3. $\mathcal{A}$ wins if and only if
 - (a) $\text{MacVer}_k(m, t) = 1$, and
 - (b) $m \notin Q$.

In that case the experiment outputs 1. Otherwise, the experiment outputs 0.

The advantage of the adversary $\mathcal{A}$ in the EUF-CMA experiment is defined to be:

$$\text{Adv}_{\text{MAC}}^{\text{EUF-CMA}}(\mathcal{A}) = \Pr[\text{G}_{\text{MAC}}^{\text{EUF-CMA}}(\mathcal{A}) = 1].$$

Definition 5. A (digital) signature scheme is a tuple of three PPT-algorithms (Gen , Sign , SigVer) such that:

1. The key generation algorithm Gen takes the security parameter 1^n and outputs a public-/private-key pair (sk, pk) : $(sk, pk) \leftarrow \text{Gen}(1^n)$.
2. The signing algorithm Sign takes as input a private key sk and a message m from some message space (that may depend on pk). It outputs the signature σ and we write this as $\sigma \leftarrow \text{Sign}_{sk}(m)$.

3. The *deterministic* verification algorithm **SigVer** takes as input a public key pk , a message m and a signature σ . It outputs a bit b , with $b = 1$ meaning *valid* and $b = 0$ meaning *invalid*. We write this as $b := \text{SigVer}_{pk}(m, \sigma)$.

We require correctness from the scheme: If $(sk, pk) \leftarrow \text{Gen}(1^n)$ then, except with negligible probability, $\text{SigVer}_{pk}(m, \text{Sign}_{sk}(m)) = 1$.

For brevity we often denote the key generation algorithms from the various primitives as $\text{Gen}()$, omitting the security parameter.

2.2 Canetti–Krawczyk Modular Design

The *modular approach*, arising originally in a 1998 paper of Bellare, Canetti and Krawczyk [3], is to first define protocols secure against a limited adversary which can then be promoted to protocols secure against a realistic adversary using a generic compiler. In the *authenticated links model* (AM) the adversary is not permitted to fabricate messages, but can otherwise control the network and deliver messages out of order or to different parties from those intended. Compilers, or *authenticators*, can be applied to messages in an AM protocol to obtain protocols in the *unauthenticated links model* (UM) where the adversary can alter or fabricate messages limited only by the available computational power.

In both the UM and AM, adversaries control the execution of protocols by initiating parties and then invoking parties with available queries, including with message inputs. (In Sect. 2.4 we describe the available adversarial queries.) Parties respond to input messages by following the protocol definition and to other queries as defined by the query. Each party computes *local output* which is available to the adversary. The local output includes protocol decisions, such as whether a message is accepted (see Sect. 2.4 for details).

Bellare et al. [3] provide a theorem showing that a secure protocol, Π_{AM} , in the AM maps to a secure protocol in the UM, Π_{UM} , if the mapping is defined by a valid authenticator. An authenticator is valid if an observer, or distinguisher, is unable to distinguish between the world where an adversary $\mathcal{A}$ is interacting with the Π_{AM} and the world where an adversary $\mathcal{U}$ is interacting with the protocol Π_{UM} . This is captured in the notion of protocol emulation in Definition 7.

Definition 6. The AM-UM distinguishing experiment, $\mathcal{G}_{\Pi_{\text{AM}}-\Pi_{\text{UM}}}^{\text{AM-UM-dist}}(\mathcal{D})$ proceeds as follows: (1) A uniform bit $b \in \{0, 1\}$ is chosen. If $b = 0$, $\mathcal{D}$ will interact with $\mathcal{A}$ and the AM protocol Π_{AM} . Otherwise, if $b = 1$, $\mathcal{D}$ will interact with $\mathcal{U}$ and the UM protocol Π_{UM} . (2) To conclude the experiment, $\mathcal{D}$ will halt and output b' . (3) The experiment will output 1 if and only if $b = b'$. We define the advantage of the distinguisher $\mathcal{D}$ to be

$$\text{Adv}_{\Pi_{\text{AM}}-\Pi_{\text{UM}}}^{\text{AM-UM-dist}}(\mathcal{D}) = 2 \cdot \left| \Pr [\mathcal{G}_{\Pi_{\text{AM}}-\Pi_{\text{UM}}}^{\text{AM-UM-dist}}(\mathcal{D}) = 1] - \frac{1}{2} \right|.$$

Definition 7. Let Π_{UM} and Π_{AM} be protocols in the UM and AM models respectively. We say that Π_{UM} ϵ -emulates Π_{AM} in unauthenticated networks if for any

UM-adversary $\mathcal{U}$ interacting with Π_{UM} , there exists an AM-adversary $\mathcal{A}$ interacting with Π_{AM} such that for any distinguisher $\mathcal{D}$ playing the AM-UM distinguishing game, $\text{Adv}_{\Pi_{\text{AM}}-\Pi_{\text{UM}}}^{\text{AM-UM-dist}}(\mathcal{D}) \leq \epsilon$.

An authenticator is a specific type of *protocol compiler* transforming one protocol into another. The modularity of the approach relies on the observation that an authenticator will actually preserve protocol security as we will see in Sect. 2.4.

Definition 8 ([3]). *An authenticator is a compiler, C , that takes an AM protocol Π_{AM} as input and outputs a UM protocol Π_{UM} , such that Π_{UM} emulates Π_{AM} .*

2.3 MT-authenticators

Defining an authenticator for any protocol, regardless of the number of messages, seems at first a difficult problem. To deal with this, BCK98 [3] define a simpler notion of an MT-authenticator, designed to authenticate a single arbitrary message. They also showed that repeated use of a valid MT-authenticator is a valid authenticator, so that protocol messages can be treated separately.

A bit more formally we define MT as a *message transmission* protocol in authenticated networks that works as follows: when P_i is activated with (P_j, m) , party P_i sends the message (P_i, P_j, m) to party P_j and outputs “ P_i sent m to P_j ”. Upon receiving (P_i, P_j, m) , P_j outputs “ P_j received m from P_i ”. Note that the quoted outputs are local outputs of the parties and are critical in proving proper emulation; however, when we later show compiled protocols we omit mention of these local outputs.

An MT-authenticator, λ , is a protocol that emulates MT in unauthenticated networks. Given a sequence of MT-authenticators, $\Lambda = (\lambda_1, \lambda_2, \dots, \lambda_t)$, the derived protocol compiler, C_Λ , uses the next MT-authenticator to authenticate the next message. More precisely, given a protocol Π in the AM with t messages, $m_1, m_2, \dots, m_t$ the protocol $\Pi' = C_\Lambda(\Pi)$ in the UM is defined as follows. For each message, m_k , sent in Π , λ_k is run to send the same message from the same initiator to the same recipient. Whenever a party, P_j , outputs “ P_j received m from P_i ” in λ_k , then Π is activated at P_j with message m_k from P_i . If Λ is a sequence of t MT-authenticators then C_Λ is an authenticator. We restate this in Theorem 1 and give a sketch of the proof, which is given in full in [3].

Theorem 1 ([3]). *Let $\Lambda = (\lambda_1, \lambda_2, \dots, \lambda_t)$ be a sequence of t MT-authenticators so that each λ_k ϵ -emulates MT. Then the compiler, C_Λ , will be an authenticator such that for any protocol Π in the AM, $C_\Lambda(\Pi)$ $(t \cdot \epsilon)$ -emulates Π in the UM.*

Proof. Let Π be an AM protocol. Let $\mathcal{U}$ be a UM-adversary interacting with $C_\Lambda(\Pi)$. $\mathcal{A}$ runs $\mathcal{U}$ on a simulated interaction. Action requests from $\mathcal{U}$ to parties in the UM can be mimicked by $\mathcal{A}$ in the AM and $\mathcal{A}$ relays its results back to $\mathcal{U}$. The only problem with the simulation could occur in the case that $\mathcal{U}$ specifies that a message is received by some party P_j from some party P_i in the UM, but

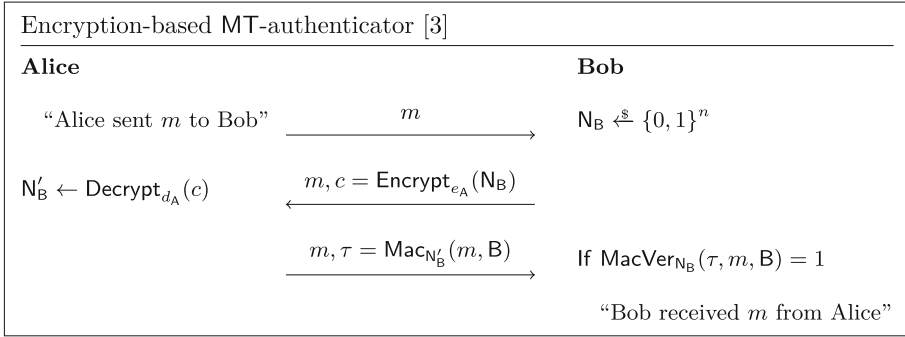


Fig. 1. Bob authenticates message m from Alice.

that message is not in the set of messages waiting for delivery in the AM. But this can happen with probability bounded by ϵ . Such an event could occur for any of the t messages and so the probability that the simulation is correct is at least $(1 - \epsilon)^t \geq 1 - t \cdot \epsilon$. Finally, any observer will be able to distinguish between the run of Π in the AM and $C_A(\Pi)$ in the UM with advantage at most $\epsilon' = t \cdot \epsilon$.

Note that although we have assumed that each MT-authenticator has the same security level ϵ , the theorem is still true if the MT-authenticators have different security levels $\epsilon_1, \epsilon_2, \dots, \epsilon_t$ and we take $\epsilon = \max_k(\epsilon_k)$. In the cases we are interested in, we will always have $t = 2$.

An example of an MT-authenticator is the encryption-based authenticator [3] shown in Fig. 1, where N_B denotes a nonce and (e_A, d_A) an encryption-decryption keypair. This is a valid MT-authenticator as long as the public key encryption used is CCA-secure and the MAC is secure. The protocol can be optimized in various ways, as we will show later.

There exist several other MT-authenticators defined in the literature including signature-based [3], MAC-based [9] and password-based [17]. We show, in compressed form, the signature-based MT-authenticator later (Fig. 8). The modular approach allows combination of any MT-authenticator together with any AM-protocol, resulting in automatically secure UM protocols. Therefore adding any new building block, either an MT-authenticator or an AM-protocol, results in several new protocols of potential interest.

2.4 SK-security

SK-security is the AKE security notion of CK01 [9], capturing session key indistinguishability and correctness of the protocol. To define this notion we need to state the capabilities of the adversary and the indistinguishability experiment. Each protocol run at a party **A** is associated with a session identifier s . In the AM the value s is an input at the start of a run to the initiator party. Later we will see that session identifiers can be replaced by protocol messages as long as parties can verify that no incomplete sessions between the same parties have the same session identifier. The state of a session consists of the following information:

- status – whether or not the session is complete, aborted, or still in progress;
- any ephemeral key material needed to complete the protocol;
- the session key, sk , if the protocol is completed and has not expired.

The global state of a party may include long-term authentication keys pk_A, sk_A . As in most AKE models, we do not explicitly model distribution of long-term keys. Furthermore, we assume that long-term public keys are immediately available to any party that needs them. This may be too strong an assumption in some real-world protocols, such as TLS, and we remark further on this issue when we examine concrete protocols in Sect. 5.

Definition 9 (Matching sessions [9]). *The two sessions $(A, B, s, role)$ and $(A', B', s', role')$ are matching if $A = B'$, $B = A'$ and $s = s'$.*

The adversary may issue the following queries, subject to certain restrictions we will see later.

- **NewSession**(A, B, s, r): the adversary issues the **NewSession** query to party A , specifying its intended partner B , the session identifier¹ s , and the role r (initiator or responder) of A in the session. A will follow the protocol definition and may return an output message intended for B .
- **Send**(A, B, m): represents activation of A by an incoming message m (possibly including a session identifier) from party B . A will follow the protocol and may reject, accept, or return an output message intended for B .
- **Corrupt**(A): the adversary learns the whole state of A including any long-term keys. The corruption event is recorded in the local output of A . Subsequently A can never be activated but the adversary can take the role of A in the protocol.
- **RevealKey**(A, B, s): the adversary learns the session key accepted in the session s by A with partner B , if it exists. The reveal event is recorded in the local output of A .
- **RevealState**(A, B, s): the adversary learns the state information associated with session s at A , such as ephemeral keys. The reveal state event is recorded in the local output of A .
- **Expire**(A, B, s): if there is a completed session s at A with B then any session key associated with that session is deleted from the memory of A . The **Expire** event is recorded in the local output of A .
- **Test**(A, B, s): this query can be asked only once and can only be made to a completed session s at A with partner B . Furthermore there cannot have been any of the following queries made: **RevealKey**(A, B, s) or **RevealState**(A, B, s) or **Corrupt**(A) or **Corrupt**(B). If the bit b specified by the challenger is $b = 1$ then the session key is returned. Otherwise $b = 0$ and a random key from the keyspace are returned.

Now we are in a position to define the SK-security experiment.

¹ We remark that instantiation of session identifiers differs between the models. In UM, s can be blank as the session identifier need not be determined by the adversary.

Definition 10. The key indistinguishability experiment, $\mathcal{G}_{\Pi}^{\text{Key-Ind}}(\mathcal{A})$ is defined as follows: (1) The challenger chooses a random bit b needed to define the Test query response. (2) The challenger initialises n parties and any long-term keys. (3) $\mathcal{A}$ may issue queries as defined above. (4) Eventually $\mathcal{A}$ halts and outputs a bit b' to indicate its guess for b , based on the response to the Test query. The experiment outputs 1 if and only if $b' = b$.

Definition 11. A key exchange protocol Π is ϵ -SK-secure if the following holds for any adversary $\mathcal{A}$:

- two honest parties (i.e. uncorrupted parties who faithfully execute the protocol instructions) completing matching sessions of the protocol Π will output the same key, except with negligible probability, and
- the advantage of the adversary $\mathcal{U}$ in the key indistinguishability experiment is: $\text{Adv}_{\Pi}^{\text{Key-Ind}}(\mathcal{A}) = 2 \cdot |\Pr[b' = b] - 1/2| \leq \epsilon$.

The final step needed to bring the modular approach together is to show that emulation preserves SK-security. This was proven in CK01 and we re-state and re-prove it as Theorem 2 including concrete bounds. Note that using emulation of an ideal key exchange process as a definition of security, the original idea of BCK98, results in too strong a definition to allow some well-known protocols to be proven secure [9, Appendix A].

Theorem 2 ([9]). Let Π be an ϵ -SK-secure protocol in the AM with t messages. Let C_{Λ} be the compiler based on MT-authenticators $\lambda_1, \lambda_2, \dots, \lambda_t$ such that for any protocol Π in the AM, $C_{\Lambda}(\Pi)$ α -emulates Π in the UM. Then protocol $\Pi' = C_{\Lambda}(\Pi)$ is an ϵ' -SK-secure protocol in the UM with $\epsilon' = \epsilon + \alpha$.

Proof. Assume to the contrary that there exists a UM adversary $\mathcal{U}$ that has advantage ϵ' in the UM. Using $\mathcal{U}$, we build an AM adversary, $\mathcal{A}$, playing the game of Definition 10. When $\mathcal{A}$ receives its setup information consisting of system parameters and public keys from its challenger, $\mathcal{A}$ sends the same information to $\mathcal{U}$. Then $\mathcal{A}$ invokes $\mathcal{U}$ and mimics its behaviour in the AM, using its challenger to respond to the action requests when any party is exposed.

When $\mathcal{U}$ sends a message to a party P_j from a party P_i in the UM, $\mathcal{A}$ sends the same message between the same parties in the AM. The emulation will be perfect unless $\mathcal{U}$ successfully sends a message m to some party P_j from P_i but m was never sent by P_i . In this case we will say that $\mathcal{U}$ made a forgery and we let **forge** be the event that a forgery happens at any time during the run of $\mathcal{U}$.

If **forge** occurs then $\mathcal{A}$ will abort the simulation and return a random bit to its challenger. Note that this also defines a distinguisher $\mathcal{D}$ which will always win in the case that **forge** occurs. If **forge** does not occur then at some point $\mathcal{U}$ will ask its Test query for a session s . $\mathcal{A}$ then announces session s for its own Test query in the AM, receives a real or random key, and returns it to $\mathcal{U}$. Eventually $\mathcal{U}$ will halt and output its bit which $\mathcal{A}$ copies as its response. In this case, $\mathcal{A}$ wins whenever $\mathcal{U}$ wins.

$$\begin{aligned}
\Pr[\mathcal{A} \text{ wins}] &= \Pr[\mathcal{A} \text{ wins} | \text{forge}] \cdot \Pr[\text{forge}] + \Pr[\mathcal{A} \text{ wins} | \neg \text{forge}] \cdot \Pr[\neg \text{forge}] \\
&= 1/2 \cdot \Pr[\text{forge}] + \Pr[\mathcal{B} \text{ wins}] \cdot (1 - \Pr[\text{forge}]) \\
&\geq \Pr[\mathcal{B} \text{ wins}] - 1/2 \cdot \Pr[\text{forge}]
\end{aligned}$$

We also implicitly defined a distinguisher, $\mathcal{D}$, which wins when `forge` occurs or wins with probability at least $1/2$ when `forge` does not occur: $\Pr[\mathcal{D} \text{ wins}] \geq \Pr[\text{forge}]/2 + 1/2$. Putting this together we get:

$$\text{Adv}_{\Pi'}^{\text{Key-Ind}}(\mathcal{U}) \leq \text{Adv}_{\Pi}^{\text{Key-Ind}}(\mathcal{A}) + \text{Adv}_{\Pi-\Pi'}^{\text{AM-UM-dist}}(\mathcal{D}).$$

2.5 Optimising the UM Protocol

Simple application of an MT-authenticator to each message of an SK-secure AM protocol results in an SK-secure UM protocol as proven in Theorem 2. However, such a protocol is far from optimal. The most obvious drawback is that a two-message protocol, such as Diffie–Hellman, compiles to a six-message protocol. The obvious way to optimise such a protocol is to “piggyback” messages going in the same direction. The resulting protocol may be secure, but formally this process may break the security proof because it may alter the order of the local output of the parties, allowing trivial distinguishability outputs of the AM protocol from the outputs of the compiled UM protocol [17].

Because of such issues, the modular approach of CK01 has been criticised [12] for not achieving efficient protocols. There is some truth in such criticisms—for example, when using signature- or encryption-based authenticators it is not possible to achieve secure 2-message AKE protocols which are often seen in the literature. Fortunately, rigorous optimisations are not difficult to achieve, typically resulting in 3-message protocols as efficient as real-world protocols. Indeed, 3-message AKE protocols are necessary in any case to achieve desirable security properties such as mutual entity authentication or key confirmation.

Hitchcock et al. [17] designed a general technique for altering message ordering in a security-preserving way. This involved defining an intermediate model between the AM and UM, which they call the *hybrid model*. Rather than use this more comprehensive approach, here we apply simple techniques to allow optimisation of the number of messages and re-use message components as session identifiers. Consequently, the drawbacks of practical application of authenticators are removed resulting in generic protocols as efficient as standalone protocols.

Compressed Authenticators. The first step is to compress the authenticator to remove redundant elements. Notice that use of the authenticator in Fig. 1 expands each message m from the AM into three messages in the UM. However, sending m in all three messages is not actually needed (to achieve security), so we can simplify the encryption-based authenticator into a compressed version shown in Fig. 2. It is not hard to see [3, 17] that removal of the repeated m fields does not affect the security of the MT-authenticator. Depending on the

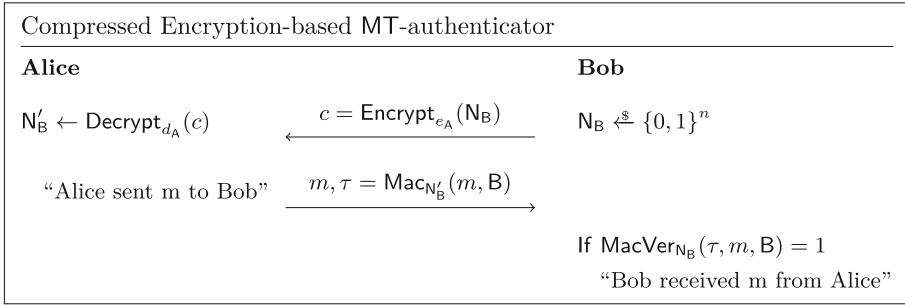


Fig. 2. Compressed version of MT-authenticator in Fig. 1.

application scenario, the version in Fig. 1 may remain appropriate. The version in Fig. 2 is useful in a situation where Bob knows that some message, as yet unknown, will be authentically received from Alice; this case typically occurs in AKE protocols. Later we will see that to apply optimisation it is important that the first message in Fig. 2 is independent of the message to be authenticated, so that it can be generated and sent early in the protocol.

Session Identifiers. In the original formulation of CK01, session identifiers are sent in each protocol run in the AM. These must be unique for each active protocol run between the same parties, but it is not defined how they should be obtained in practice. Although the only property required of session identifiers is uniqueness, a natural way of obtaining them is to use random values chosen by each party; in that case the probability that session identifiers are not unique is negligible. In practice it may not be a burden for each party to ensure that there are no other incomplete sessions with the same identifier so that uniqueness is unconditionally guaranteed.

We assume that higher communication layers will provide a mechanism to ensure that messages get delivered to the correct session. They can also be explicitly added to the protocol messages if desired.

3 KEM-Based Building Blocks

This section defines and proves security for the basic KEM-based MT-authenticator and AM protocol, which will be brought together in Sect. 4 as components in defining generic efficient KEM-based AKE.

3.1 KEM-Based MT-authenticator

Figure 3 illustrates our KEM-based MT-authenticator. The construction is closely related to the encryption-based authenticator of BCK98.

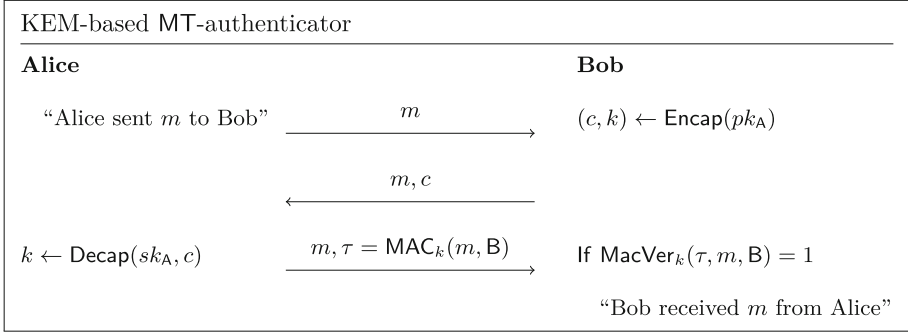


Fig. 3. KEM-based MT-authenticator, λ_{KEM} : Bob authenticates m from Alice.

Next we give a theorem that λ_{KEM} is secure, meaning that it emulates MT in unauthenticated networks, as long as the KEM used achieves CCA security. The proof of Theorem 3 follows the proof strategy of Bellare et al. [3, Prop. 5] for their encryption-based authenticator. The complete proof is given in the full version [8].

Theorem 3. *The KEM-based MT-authenticator, λ_{KEM} , in Fig. 3, when instantiated with a CCA-secure KEM and a secure MAC scheme, ϵ -emulates protocol MT in unauthenticated networks such that $\epsilon \leq l \cdot (\text{Adv}_{\text{KEM}}^{\text{CCA}}(\mathcal{D}) + \text{Adv}_{\text{MAC}}^{\text{EUF-CMA}}(\mathcal{F}))$ where $l = n_P^2 \times n_M$, n_P is the number of parties that run the protocol and n_M is the maximum number of challenge messages that can be sent by any party.*

Now that λ_{KEM} is proven to be an MT-authenticator we can invoke Theorem 2 to conclude that λ_{KEM} can be used to authenticate messages in an SK-secure AM protocol and results in a SK-secure UM protocol. In order to optimise the resulting protocol we will want to use a compressed version of the authenticator (see Sect. 2.5) as shown in Fig. 4.

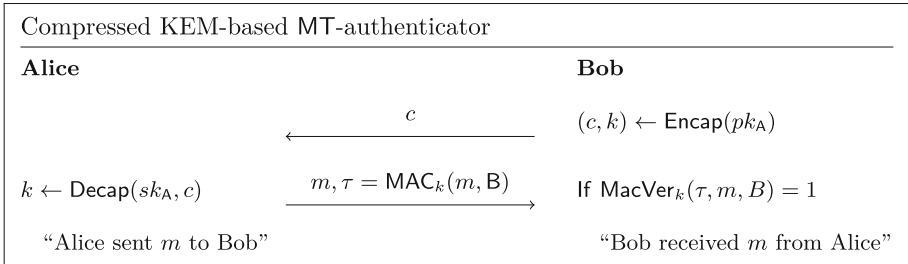


Fig. 4. λ_{KEM} , the compressed KEM-based MT-authenticator.

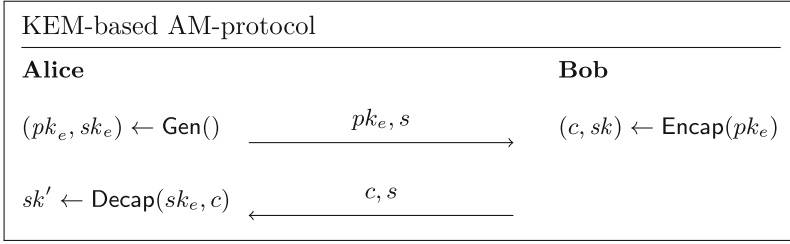


Fig. 5. KEM-based protocol with any CPA-secure KEM (see Definition 2).

The security proof for the compressed authenticator is identical to the proof for the full authenticator since the only difference is the deletion of plaintext messages in the UM which are ignored in the security proof.

Corollary 1. *Theorem 3 still holds if the authenticator in Fig. 3 is replaced by the compressed KEM-based MT-authenticator, λ_{KEM} , in Fig. 4.*

3.2 KEM-Based AM Protocol

In Fig. 5 we present a KEM-based protocol Π that is SK-secure in the AM. The protocol is a generalisation of the basic Diffie–Hellman AM protocol of CK01 [9]. We assume that a setup with parameters for the KEM is known already to all parties. The initiator A will be invoked by the $\text{NewSession}(A, B, s, r)$ query and responds with a new ephemeral KEM public key pk_e . Upon receipt of (pk_e, s) the responder encapsulates a new session key sk in c , and returns it to party A.

Theorem 4. *Let $\mathcal{A}$ be an adversary against the SK-security of protocol Π shown in Fig. 5. Let $\mathcal{A}$ interact with at most q sessions of Π for each pair of parties. Let n be the maximum number of parties involved in the protocol run. Then the advantage of $\mathcal{A}$ can be bounded by: $\text{Adv}_{\Pi}^{\text{SK}}(\mathcal{A}) \leq n^2 q \cdot \text{Adv}_{\text{KEM}}^{\text{CPA}}(\mathcal{B})$.*

The proof of Theorem 4 is in the full version [8].

4 Generic KEM-Based AKE Protocols

With the building blocks from Sect. 3 we now apply MT-authenticators to AM protocols and optimise them to obtain protocols which are both SK-secure in the realistic UM security model and efficient in comparison with other protocols in the literature. There is no restriction to apply the new MT-authenticator in Fig. 4 only to the new AM protocol in Fig. 5; the authenticator can be applied to any SK-secure AM protocol and any authenticator can be applied to the KEM-based AM protocol. Furthermore, we may apply different MT-authenticators to each of the messages in an AM protocol [17, Theorem 6] resulting in yet more ways to construct different secure protocols.

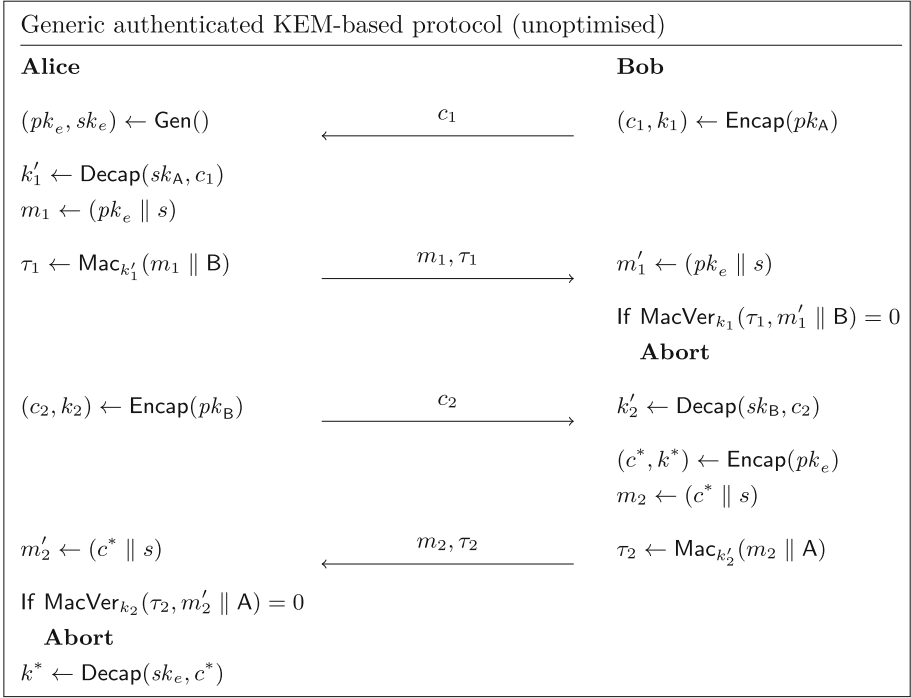


Fig. 6. Generic 4-message protocol obtained by compiling the KEM-based AM protocol with the compressed KEM-based MT-authenticator.

Due to our field's focus on post-quantum security in recent years, we emphasise KEM-based and signature-based components in this section, allowing us to apply any of the primitives from the NIST competition library. We illustrate this usage with several different examples in this section, applying both our new KEM-based authenticator and the existing signature-based authenticator to achieve a variety of protocols. Another example, also with potential for post-quantum security, is to apply the MAC-based authenticator of CK01 to our KEM-based AM protocol. This results in a protocol suitable for pre-shared key environments which is a common scenario, for example in TLS and IPsec. Details of a MAC-based generic protocol construction are available in Appendix 3.

4.1 Compiled KEM-Based Protocol and Optimization

We start with the AM-secure protocol from Fig. 5 and then apply the compiler consisting of application of the compressed MT-authenticator to each of its two messages. This leads to the 4-message protocol of Fig. 6.

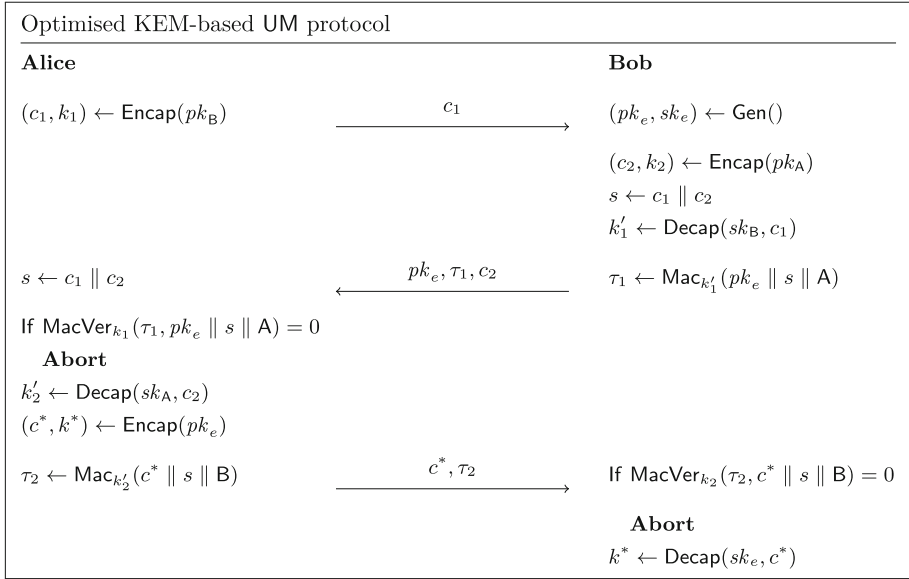


Fig. 7. Optimised UM protocol from the KEM-based AM protocol and the KEM-based MT-authenticator.

Messages 1 and 2 are the result of applying the compressed MT-authenticator to authenticate the ephemeral public key pk_e generated by Alice. Messages 3 and 4 are the result of applying the compressed MT-authenticator to authenticate the encapsulated shared key c^* generated by Bob. The difference between m_1 and m'_1 (resp. m_2 and m'_2) in Fig. 6 is that both players have their own version of s —the MAC verifies the integrity of both the message and the session.

To optimise the 4-message protocol in Fig. 6 we take four simple steps: (1) The messages that were numbered 2 and 3 will be sent in parallel as a new message with number 2. This does not change the order or contents of any messages. (2) The session identifier, s , will be constructed by the parties as part of the protocol, instead of taking it as an external input to the protocol. Recall that the only requirements on s are to be unique between the parties amongst any incomplete protocol session between the two parties. We choose $s = c_1 \parallel c_2$ where c_1 and c_2 are the (randomised) encapsulations (ciphertexts) generated by each party. (3) Repeated message fields and fields previously generated by message receivers are removed from messages. (4) The protocol parties are re-labelled so that Alice becomes the protocol initiator. Combining all of these steps we obtain the optimised protocol shown in Fig. 7.

As far as we are aware, the precise protocol of Fig. 7 is new in the literature. There are several existing protocols also aimed at achieving AKE based only on KEMs [15, 18, 23] or encryption [12]. Several of these are motivated by the desire to avoid signatures, which tend to suffer efficiency disadvantages compared with KEMs in the post-quantum examples from the NIST competition. The security

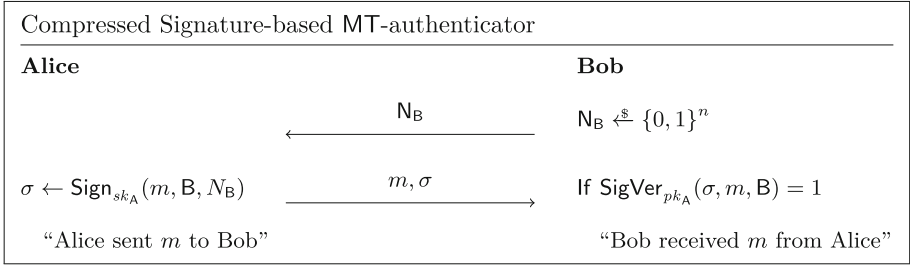


Fig. 8. λ_{Sign} , a compressed signature-based MT-authenticator.

varies between of each these protocols. For example, the FSXY protocol [15] provides security against ephemeral key leakage whereas KEMTLS [23], like the protocol of Fig. 7, lacks this property. On the other hand, our protocol does allow state reveals from non-target sessions. KEMTLS is also designed to provide only one-way (server) authentication. Making a judgement on which of these protocols is “better” is therefore difficult since it depends on the security requirements and implementation details. In Sect. 5 we compare efficiency using concrete KEMs and signature schemes to get a better feel for the relative efficiencies.

4.2 Generic Protocols Using Signatures

We now look at two further generic protocols which we can obtain by using signatures in combination with our KEM-based AM protocol. We will need to apply the compressed signature-based authenticator shown in Fig. 8.

The authenticator λ_{Sign} is derived from the authenticator of BCK98 by removing the unnecessary message components in exactly the same way as for the encryption- and KEM-based authenticators. As before, the existing proof that the full authenticator is a valid MT-authenticator [3] still holds.

The optimised protocol for the KEM-based AM protocol compiled with the signature-based authenticator is shown in Fig. 9. The optimisation follows the same process as described in Sect. 4.1. Although more general, the resulting protocol has much in common with the signed Diffie–Hellman protocol which has been widely known and deployed for many years and is today the usual AKE in the latest version of TLS (though with only one-sided authentication).

We have another way to authenticate the KEM-based AM protocol, namely to authenticate its two messages with different MT-authenticators. As far as we are aware there are no examples of such a protocol in the existing literature. There can be practical usages, for example when signatures are very expensive to generate but very cheap to verify. In such a case, when a powerful server authenticates its AM message it can shift computation away from a lightweight client by using the signature-based authenticator, while the client can avoid generating signatures by using a different KEM-based authenticator. In Fig. 10 we show the optimised protocol using the KEM-based authenticator for the

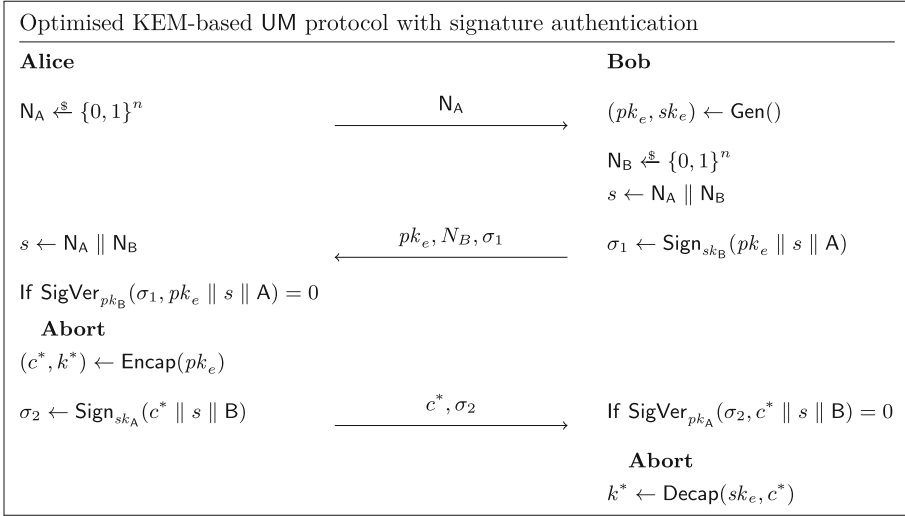


Fig. 9. Optimised UM protocol from the KEM-based AM protocol and the signature-based MT-authenticator.

first message and the signature-based authenticator for the second message. A mirror protocol results from using the two authenticators the other way around. For completeness this optimised protocol is given as Fig. 13 in Appendix 2.

5 Concrete Post-quantum Secure AKE Protocols

In the previous section we have presented optimised generic AKE protocols which will be secure as long as the KEM, signature and MAC primitives are instantiated with secure instances. Even restricting to a handful of currently best-trusted post-quantum primitives, this leads to hundreds of potential concrete protocols, bearing in mind that we have shown that different KEMs and signatures can be mixed in the same protocol and observing that the generic protocols are not symmetric between initiator and responder. The question of whether the concrete instantiated protocols are practical in terms of computational efficiency and message size is a natural one.

5.1 Computational Cost

To give an impression of the computational costs of our new protocols we summarize the number of public key operations needed in each of our optimised protocols in the upper part of Table 1. The lower part of the same table includes the number of similar operations for some prominent existing protocols.

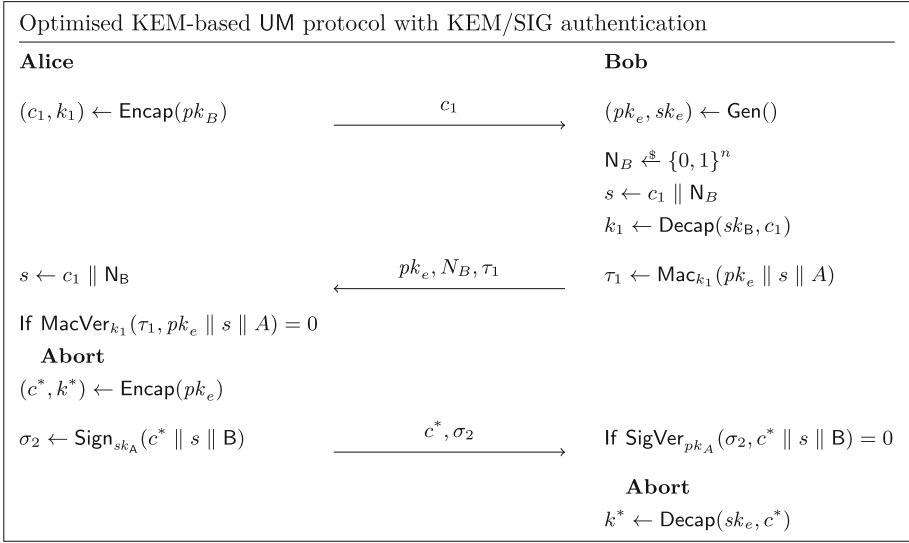


Fig. 10. Optimised UM protocol from the KEM-based AM protocol and the KEM-based MT-authenticator used for the first message, and the signature-based MT-authenticator for the second message.

Table 1. The number of public key operations for ours and existing protocols.

	Initiator					Responder				
	Gen _e	Encap	Decap	Sign	SigVer	Gen _e	Encap	Decap	Sign	SigVer
Figure 7. KEM/KEM	0	2	1	0	0	1	1	2	0	0
Figure 9. Sig/Sig	0	1	0	1	1	1	1	0	1	1
Figure 10. KEM/Sig	0	2	0	1	0	1	0	2	0	1
Figure 11. Sig/KEM	0	1	1	0	1	1	1	1	1	0
Figure 13. MAC/MAC	0	1	0	0	0	1	0	1	0	0
TLS 1.3 ^a	1	0	1	0	1	0	1	0	1	0
KEMTLS ^b [23]	1	1	1	0	0	0	1	1	0	0
KEMTLS-pdk [24]	1	1	2	0	0	0	2	1	0	0
PQ-WireGuard [18]	1	1	2	0	0	0	2	1	0	0
SSW17 ^c [12]	1	1 ^d	2	0	0	0	2	1	0	0

^a Using Diffie-Hellman as an ephemeral KEM. Unilateral authentication

^b Unilateral authentication

^c Assuming that our KEM-based AM protocol is used as the base protocol.

^d Encryption is needed in the full protocol, not encapsulation

All of the protocols in Table 1 use three passes and three rounds. However, they do not all have the same goals or assumptions. TLS and KEMTLS only aim for server-side authentication while our protocol in Fig. 13 assumes pre-shared keys. PQ-WireGuard [18] is a variant of the WireGuard protocol using

only KEMs. Its design is based on the FSXY protocol [15]. All of the protocols in the lower half of Table 1 use only KEMs, both for authentication and key exchange. When comparing with our KEM-only protocol of Fig. 7 we see that the main computational effort is the same as in the three bottom protocols which are all KEM-only protocols. We conclude that our protocols are comparable in computation to existing ones. Another difference between the various protocols is on which side most computations are performed, e.g. in Fig. 10 the initiator Alice encapsulates twice while Bob performs computationally heavier decapsulations and generation of the ephemeral key.

The most obvious difference between the upper and lower part of the table is that our designs have the responder generating the ephemeral KEM key while all existing protocols shown give this task to the initiating party. We do not believe that either option is inherently better, rather it depends on the relative costs of generation, encapsulation and decapsulation of the instantiating ephemeral KEM. For some well-known KEMs (Table 3a), key generation is far more costly than encapsulation or decapsulation. To minimise the overall protocol cost to both parties it seems better to use an algorithm with more uniform cost for the three KEM operations, but if it is desired to reduce the cost of one party at the expense of the other then different algorithms can be better.

It can be argued that implementation is most efficient when the same concrete KEM is used for all three of the KEM instances in the all-KEM protocols. This should be true at least with regard to the codebase needed in any implementation. However, this may not be the case when it comes to counting computation cycles. Recall that the AM protocol includes generation of an ephemeral public key, while the long-term keys are generated only once before the protocol runs. Therefore it can make sense to use a KEM with an efficient key generation algorithm for the ephemeral KEM, and a different one with a much less efficient key generation algorithm for the KEM using the long-term keys. PQ-WireGuard [18] does exactly this, using Classic McEliece for the long-term KEM and a variant of Saber for the ephemeral KEM. The size of its public key (Table 3a) shows why using Classic McEliece for the ephemeral KEM seems to be a bad idea.

Current known post-quantum signatures tend to be computationally less efficient than KEM constructions (Table 3b) where signing is much more expensive than decapsulation in known algorithms. It is therefore natural that KEM-based authentication currently is seen favourably. This can change in the future, and the NIST focus on new post-quantum signature proposals may well lead to more efficient post-quantum secure signature algorithms. To our knowledge, there is no analog to our KEM/Sig or Sig/KEM protocols in the literature, neither are we aware of post-quantum proposals for the pre-shared key case.

5.2 Communications Cost

In Table 2 we take an inventory of the message fields in each of our abstract protocols. Due to the optimisation techniques explained earlier, the number of fields sent and received by each party is three in all cases. Informally, at least, this is a minimum since the ephemeral public key needs to be communicated and

Table 2. What comprises the messages sent in each protocol.

	Message 1	Message 2	Message 3
Figure 7. KEM/KEM	ct	pk, ct, MAC	ct, MAC
Figure 9. Sig/Sig	N	pk, N, σ	ct, σ
Figure 10. KEM/Sig	ct	pk, N, MAC	pk, MAC
Figure 11. Sig/KEM	N	pk, σ, ct	ct, MAC
Figure 13. MAC/MAC	N	pk, N, MAC	ct, MAC

then used in the response, and each of these two messages must be authenticated using a fresh value chosen by the other party.

The size of these fields depends on the parameters of the concrete primitives chosen. In July 2022, NIST announced a first list of selected candidates as a result of its Post-Quantum Cryptography competition [1], pointing out CRYSTALS-Kyber as their selected KEM and CRYSTALS-Dilithium as their selected signature algorithm. Using the real-world efficiency of the Kyber KEM and the Dilithium signature scheme, in Tables 3a and 3b and naively adding up these numbers, all messages in our Fig. 10 protocol would be under 5 kB for Kyber-1024, which definitely is practical. Another look at the ephemeral public key sizes in Table 3a shows why the choice of Saber in PQ-WireGuard [18] is an obvious one. We note that before its recent demise, SIKE looked an even more promising candidate to minimise the ephemeral key size.

Just as for computation efficiency, currently accepted post-quantum secure signature candidates do not look attractive for communications efficiency as shown in Table 3b. To minimise signature size FALCON is a better choice than Dilithium, but requires a trade-off with computation.

We reiterate that Table 2 assumes that authentic long-term public keys are available to all parties by some external channel. This fits some real-world protocols (such as WireGuard) but not others (such as TLS). Post-quantum signatures used to certify the long-term public keys can be chosen independently of other concrete choices in the protocol. This choice will obviously affect both the computation for each party and the size of the protocol messages. Although registration of public keys can avoid use of post-quantum signatures [16], it seems necessary to use signatures to achieve usual certificate properties.

6 Conclusion and Future Work

As summarized in Sect. 1.2, the main contributions of this work are the new KEM-based authenticator and corresponding security proof, the new proofs in the AM-model and the derivation of several new generic AKE protocols. We hope that the flexibility of protocol designs can be useful in fitting AKE protocols to different application use cases and that as new concrete KEMs and signature schemes are developed the generic protocols will yield new and interesting instantiations. Some of the ways to extend the work are the following.

- Adding additional security properties; for example, application of the twisted-PRF trick [15] can likely be added to an AM-protocol to secure against ephemeral leakage.
- Check whether use of hybrid-KEMs (secure against conventional adversaries based on traditional assumptions and secure against post-quantum adversaries based on new assumptions) can be usefully applied to obtain hybrid-secure AKE [6].
- Since the modular approach does not naturally lead to tight reductions, it would be useful to improve on this, although in the end it may be necessary to complement new protocol designs obtained from the modular approach with a monolithic proof in a stronger model for some concrete protocol.
- Applying an authenticator just to one message (from a two-message AM protocol) will allow for unilateral authentication such as is common in TLS. The security and efficiency of such a generic protocol deserves analysis.
- Real-world implementations of the generic protocols is also left to future work—obviously this would be a prerequisite for future adaptation, and give us a more concrete comparison of their efficiency in the real world.

Appendix 1: ETSI Tables

Tables 3a and 3b present the computational efficiency of various KEMs and signatures from the NIST competition. The figures are taken from two recent reports from ETSI [25, 26]. They are not intended as definitive efficiency comparisons—indeed some of the figures have already been improved upon—but rather to illustrate typical ballpark figures and highlight the big variation between many of the existing proposals.

We find it interesting to remark on a major difference regarding the *symmetry* of the computation requirements between Diffie–Hellman and the AM protocol (Fig. 5) which can be regarded as a generalisation. The computational requirements for Diffie–Hellman are the same for both initiator and responder. In the AM protocol the initiator runs **Gen** and **Decap** while the responder runs only **Encap**. Of itself this is not significant, since **Encap** really has two purposes: to generate the new key for the responder and to generate the encapsulation for the initiator. Thus in the Diffie–Hellman case the cost of **Encap** is the same as the cost of **Gen** plus the cost of **Decap**. However, in all the examples in Table 3a this is nowhere close to being true. Indeed **Encap** is always significantly cheaper than **Gen** plus **Decap**, which may be important when deciding which party take the role of initiator in a protocol run.

Table 3. Confusion matrix of one of the folds for the final classification of DGCNN-MS-T-W, with $W = 150$

(a) The efficiency of various KEMs [25].						
	Gen	Encap	Decap	pk	ct	NIST security category
mceliece348864	36641040	44 350	134 745	261120	128	1
mceliece460896	117067765	117 782	271 694	524160	188	3
KYBER512	33856	45 200	59 088	800	768	1
KYBER1024	73544	97 324	115 332	1568	1568	3
ntruhs2048677	309216	83 519	59 729	930	930	1
ntruhs4096821	431667	98 809	75 384	1230	1230	3
LightSaber	45152	49 948	47 852	672	736	1
Saber	66727	79 064	76 612	992	1088	3

(b) The efficiency of various signature schemes [26].				
	Sign	SigVer	σ	NIST security category
Dilithium-3	269 000	118 000	3293	3
Dilithium-5	429 000	179 000	4595	5
FALCON-512	386 678	82 340	666	1
FALCON-1025	789 564	168 498	1280	5

Appendix 2: Optimised KEM-Based UM Protocol with SIG/KEM Authentication

We give here in Fig. 11 an optimized protocol using the signature-based authenticator for the first message and the KEM-based authenticator for the second. This is reversed from the protocol in Fig. 10.

Appendix 3: MAC-Based MT-Authenticator

Canetti and Krawczyk [9] present also a MAC-based MT-authenticator (interestingly described only in compressed form) as shown in Fig. 12. This authenticator can be useful in scenarios where pre-shared keys exist such as in many use-cases of TLS with lightweight entities and also in session resumption in the latest TLS 1.3 version.

Since MACs are expected to remain secure in the post-quantum setting it makes sense to combine this authenticator with our KEM-based AM protocol to obtain a post-quantum secure AKE protocol suitable for pre-shared key applications. Figure 13 show the resulting optimised protocol.

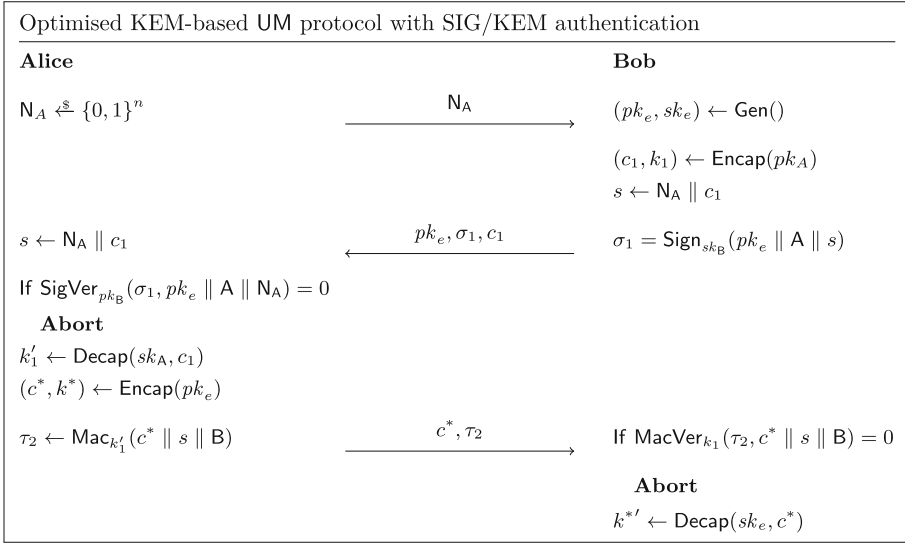


Fig. 11. Optimised UM protocol from the KEM-based AM protocol and the signature-based MT-authenticator used for authenticating the first message, and the kem-based MT-authenticator authenticating the second message.

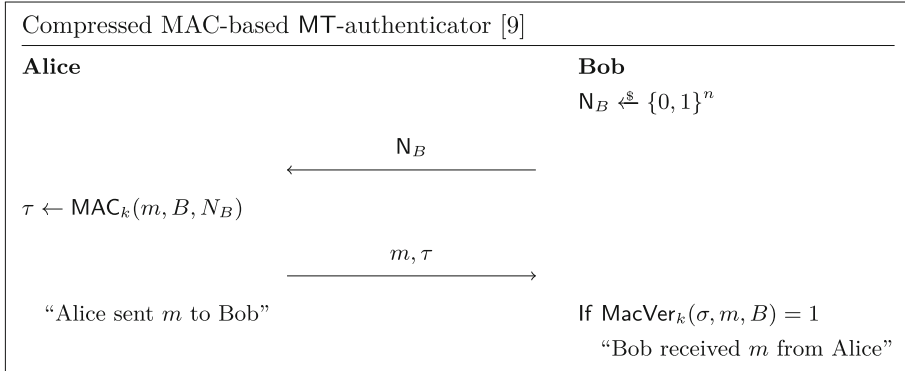


Fig. 12. λ_{MAC} , a compressed MAC-based MT-authenticator with shared key k .

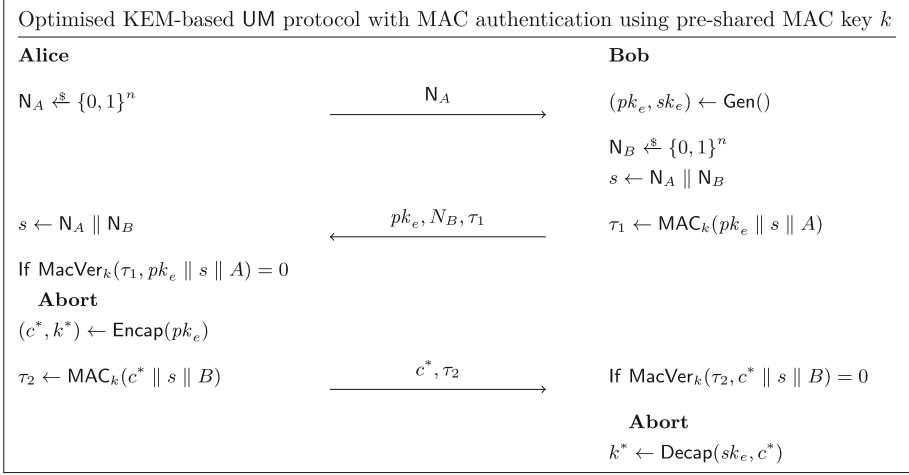


Fig. 13. Optimised UM protocol from the KEM-based AM protocol and the MAC-based MT-authenticator.

References

1. Alagic, G., et al.: Status report on the third round of the NIST post-quantum cryptography standardization process. Technical report, National Institute of Standards and Technology (2022). <https://csrc.nist.gov/publications/detail/nistir/8413/final>
2. Angel, Y., Dowling, B., Hülsing, A., Schwabe, P., Weber, F.: Post quantum noise. Cryptology ePrint Archive, Report 2022/539 (2022). <https://eprint.iacr.org/2022/539>
3. Bellare, M., Canetti, R., Krawczyk, H.: A modular approach to the design and analysis of authentication and key exchange protocols (extended abstract). In: 30th ACM STOC, pp. 419–428. ACM Press (1998)
4. Bergsma, F., Jager, T., Schwenk, J.: One-round key exchange with strong security: an efficient and generic construction in the standard model. In: Katz, J. (ed.) PKC 2015. LNCS, vol. 9020, pp. 477–494. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46447-2_21
5. Bernstein, D.J., Lange, T.: Post-quantum cryptography. Nature **549**(7671), 188–194 (2017)
6. Bindel, N., Brendel, J., Fischlin, M., Goncalves, B., Stebila, D.: Hybrid key encapsulation mechanisms and authenticated key exchange. In: Ding, J., Steinwandt, R. (eds.) PQCrypto 2019. LNCS, vol. 11505, pp. 206–226. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-25510-7_12
7. Bos, J.W., et al.: Frodo: take off the ring! Practical, quantum-secure key exchange from LWE. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) ACM CCS 2016, pp. 1006–1018. ACM Press (2016)
8. Boyd, C., de Kock, B., Millerjord, L.: Modular design of KEM-based authenticated key exchange. Cryptology ePrint Archive, Paper 2023/167 (2023). <https://eprint.iacr.org/2023/167>

9. Canetti, R., Krawczyk, H.: Analysis of key-exchange protocols and their use for building secure channels. In: Pfitzmann, B. (ed.) EUROCRYPT 2001. LNCS, vol. 2045, pp. 453–474. Springer, Heidelberg (2001). https://doi.org/10.1007/3-540-44987-6_28
10. Celi, S., et al.: Implementing and measuring KEMTLS. Cryptology ePrint Archive, Report 2021/1019 (2021). <https://eprint.iacr.org/2021/1019>
11. Cremers, C.: Examining indistinguishability-based security models for key exchange protocols: the case of CK, CK-HMQV, and eCK. In: Cheung, B.S.N., Hui, L.C.K., Sandhu, R.S., Wong, D.S. (eds.) ASIACCS 2011, pp. 80–91. ACM Press (2011)
12. de Saint Guilhem, C., Smart, N.P., Warinschi, B.: Generic forward-secure key agreement without signatures. In: Nguyen, P., Zhou, J. (eds.) ISC 2017. LNCS, vol. 10599, pp. 114–133. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-69659-1_7
13. Ding, J., Alsayigh, S., Lancrenon, J., RV, S., Snook, M.: Provably secure password authenticated key exchange based on RLWE for the post-quantum world. In: Handschuh, H. (ed.) CT-RSA 2017. LNCS, vol. 10159, pp. 183–204. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-52153-4_11
14. Ding, J., Xie, X., Lin, X.: A simple provably secure key exchange scheme based on the learning with errors problem. Cryptology ePrint Archive, Paper 2012/688 (2012). <https://eprint.iacr.org/2012/688>
15. Fujioka, A., Suzuki, K., Xagawa, K., Yoneyama, K.: Strongly secure authenticated key exchange from factoring, codes, and lattices. In: Fischlin, M., Buchmann, J., Manulis, M. (eds.) PKC 2012. LNCS, vol. 7293, pp. 467–484. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-30057-8_28
16. Güneysu, T., Hodges, P., Land, G., Ounsworth, M., Stebila, D., Zaverucha, G.: Proof-of-possession for KEM certificates using verifiable generation. Cryptology ePrint Archive, Report 2022/703 (2022). <https://eprint.iacr.org/2022/703>
17. Hitchcock, Y., Boyd, C., Nieto, J.M.G.: Modular proofs for key exchange: rigorous optimizations in the Canetti-Krawczyk model. Appl. Algebra Eng. Commun. Comput. **16**(6), 405–438 (2006)
18. Hülsing, A., Ning, K.-C., Schwabe, P., Weber, F., Zimmermann, P.R.: Post-quantum WireGuard. Cryptology ePrint Archive, Report 2020/379 (2020). <https://eprint.iacr.org/2020/379>
19. Jager, T., Kiltz, E., Riepel, D., Schäge, S.: Tightly-secure authenticated key exchange, revisited. In: Canteaut, A., Standaert, F.-X. (eds.) EUROCRYPT 2021. LNCS, vol. 12696, pp. 117–146. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-77870-5_5
20. Katz, J., Lindell, Y.: Introduction to Modern Cryptography, 2nd edn. CRC Press, Boca Raton (2014)
21. Krawczyk, H.: HMQV: a high-performance secure Diffie-Hellman protocol. In: Shoup, V. (ed.) CRYPTO 2005. LNCS, vol. 3621, pp. 546–566. Springer, Heidelberg (2005). https://doi.org/10.1007/11535218_33
22. Peikert, C.: A decade of lattice cryptography. Cryptology ePrint Archive, Paper 2015/939 (2015). <https://eprint.iacr.org/2015/939>
23. Schwabe, P., Stebila, D., Wiggers, T.: Post-quantum TLS without handshake signatures. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) ACM CCS 2020, pp. 1461–1480. ACM Press (2020)

24. Schwabe, P., Stebila, D., Wiggers, T.: More efficient post-quantum KEMTLS with pre-distributed public keys. In: Bertino, E., Shulman, H., Waidner, M. (eds.) ESORICS 2021. LNCS, vol. 12972, pp. 3–22. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-88418-5_1
25. ETSI Technical Committee Cyber Security. Quantum-safe public-key encryption and key encapsulation. ETSI TR 103823, ETSI (2021)
26. ETSI Technical Committee Cyber Security. Quantum-safe signatures. ETSI TR 103616, ETSI (2021)